GPU Acceleration of Batched Tensor-Times-Vector Operations

DuBose Tuller
CSC 347/647 — Spring 2026

April 2026

Abstract

The batched tensor-times-vector (BTTV) operation contracts one mode of tensor with a batch of vectors and is a major computational component for using Newton's method for computing the CP tensor decomposition. We implement all six mode-contraction variants in three ways: a serial C baseline, an OpenMP multi-threaded CPU version, and a CUDA GPU version that supports tensors larger than GPU memory via adaptive K-chunking. On a $1000 \times 1000 \times 1000$ tensor, the GPU achieves $7.8\times$ to $30.3\times$ speedup over the serial baseline depending on the contraction case, outperforming 32-thread OpenMP on every case. Memory access patterns prove to be the dominant performance factor: cases with strided memory access that cripple CPU caches are handled efficiently by the GPU's high-bandwidth memory subsystem.

1 Introduction

My current research involves applying Newton's method to compute the CANONDECOMP/PARAFAC (CP) tensor decomposition. The CP decomposition factorizes an d-way tensor into a sum of rank-1 components and appears throughout signal processing, chemometrics, and machine learning. A key inner loop of the Newton-method formulation requires computing a *batched tensor-times-vector* (BTTV) product for each of the three modes of the tensor at every iteration.

The goal of this project is to port the BTTV operation to a GPU and measure the speedup over a serial CPU baseline. We also compare against an OpenMP multi-threaded version to contextualize where the GPU's advantage comes from. The serial starting point is derived from a MATLAB MEX implementation that originally called BLAS routines (`dgemv`/`ddot`); we replace those with plain nested loops to obtain a pedagogically transparent baseline that can be ported directly to CUDA.

All three implementations read from the same pre-generated binary data file so that timing comparisons reflect identical inputs. Experiments were run on the Wake Forest University HPC cluster using an NVIDIA RTX 2080Ti GPU (approximately 8 GB of VRAM available) and a multi-core CPU node with up to 32 OpenMP threads. For all of our experimentation, we focus on three-way tensors.

2 The Batched Tensor-Times-Vector Problem

2.1 Tensor layout and notation

A third-order tensor $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$ is stored in column-major order, so element (i, j, k) resides at

```
X[i + j*I + k*I*J].
```

A *mode-m tensor-times-matrix* (TTM) product contracts mode m of $\mathcal{X}$ with a matrix, reducing the tensor by one dimension. The *batched* variant processes a family of vectors simultaneously, batching over a second mode $n \neq m$, and produces a matrix output.

2.2 The six contraction cases

For a third-order tensor there are six ordered pairs (m, n) with $m, n \in \{1, 2, 3\}$ and $m \neq n$. Table 1 lists the index formula for each case. In every case the output matrix Y has one entry per (b, c) pair, where b is the batch index (mode n) and c is the remaining free index (the third mode), and each entry is a dot product of length equal to the contracted mode dimension.

Table 1: All six BTTV cases for a third-order tensor.

m	n	Output formula	Y size
1	3	$Y[j{+}kJ] = \sum_i X[i{+}jI{+}kIJ] \cdot U[i{+}kI]$	$J \times K$
1	2	$Y[k{+}jK] = \sum_i X[i{+}jI{+}kIJ] \cdot U[i{+}jI]$	$K \times J$
2	3	$Y[i{+}kI] = \sum_j X[i{+}jI{+}kIJ] \cdot U[j{+}kJ]$	$I \times K$
3	2	$Y[i{+}jI] = \sum_k X[i{+}jI{+}kIJ] \cdot U[k{+}jK]$	$I \times J$
2	1	$Y[k{+}iK] = \sum_j X[i{+}jI{+}kIJ] \cdot U[j{+}iJ]$	$K \times I$
3	1	$Y[j{+}iJ] = \sum_k X[i{+}jI{+}kIJ] \cdot U[k{+}iK]$	$J \times I$

Every case performs exactly $O(IJK)$ multiply-accumulate operations, so any difference in wall-clock time between cases is entirely attributable to memory access patterns.

2.3 Data generation

A separate utility (`generate/generate.c`) writes a reproducible random tensor and vector buffer to a binary file: three `int32` header values (I, J, K), followed by X as IJK floats in column-major order, followed by U as $\max(IJ, IK, JK)$ floats (sufficient for all six cases). The generator uses `srand(42)` so every run produces identical data. All three benchmark programs (`serial`, `batched_ttv_omp`, `batched_ttv`) consume the same file, guaranteeing a fair comparison.

3 Serial Algorithm

3.1 Structure

The serial implementation (`serial/batched_ttv.c`) provides one function per case. All six have the same structure: two nested loops over the two output dimensions, with an inner loop that accumulates the dot product into a thread-local scalar before writing to Y. Algorithm 1 shows the case (m=1, n=3) as a representative example.

The code is compiled with `gcc -Wall` (no explicit optimization flag, so `-O0`) to provide a straightforward baseline. At `-O0` the compiler performs no optimization: every intermediate value is spilled to a stack slot between statements, so each multiply-accumulate in the inner loop requires multiple extra loads and stores beyond the actual `X` and `U` reads. The accumulator is never kept in a register across iterations. The practical consequence is that even the most cache-friendly cases achieve only about 1.2 GB/s of effective memory throughput — roughly 2% of DDR4 peak — because instruction overhead, not data movement, dominates each iteration. Timing uses `clock_gettime(CLOCK_MONOTONIC)` bracketing each function call.

noend 1 Serial BTTV, case (m=1, n=3)

```
for k = 0 ... K−1 do
    for j = 0 ... J−1 do
        s ← 0
        for i = 0 ... I−1 do
            s += X[i+jI+kIJ] · U[i+kI]
        Y[j+kJ] ← s
```

3.2 Memory access patterns

Because $\mathcal{X}$ is column-major, the stride of the innermost loop access depends on which mode is contracted. Table 2 shows the inner- and middle-loop strides for all six cases and the measured wall-clock time at $I = J = K = 1000$.

Table 2: Memory access strides and serial timing at $n = 1000$.

Case (m, n)	Inner stride	Middle stride	Time (s)
$(1, 3)$	1	I	3.33
$(1, 2)$	1	IJ	3.37
$(2, 3)$	I	1	3.37
$(3, 2)$	IJ	1	3.48
$(2, 1)$	I	IJ	6.84
$(3, 1)$	IJ	I	9.72

Cases $(1, 3)$ and $(1, 2)$ contract over mode 1, which has stride 1 in memory, so the innermost loop accesses X sequentially. The hardware prefetcher handles this well, and these cases run in about 3.3 s. Case $(3, 1)$ contracts over mode 3, which has stride $IJ = 10^6$ (a 4 MB jump per element at single precision), thrashing the cache and taking nearly 10 s despite identical FLOP count.

3.3 Arithmetic intensity and the bandwidth-bound regime

The timing differences above are a symptom of a deeper property: BTTV has extremely low *arithmetic intensity* (AI), defined as FLOPs performed per byte of data read. For a single output element, the inner loop of length L (the contracted mode dimension) reads L values from X and L values from U, performs $2L$ multiply-accumulate operations, and writes one float:

$$\mathrm{AI} = \frac{2L \text{ FLOPs}}{8L + 4 \text{ bytes}} \approx 0.25 \text{ FLOP/byte}.$$

This is well below the *ridge point* of any modern processor — the minimum AI at which a chip becomes compute-bound rather than bandwidth-bound. For both the RTX 2080Ti and a typical server CPU the ridge point is in the range of 15–20 FLOP/byte. At 0.25 FLOP/byte, BTTV will always be bottlenecked by memory bandwidth, never by floating-point throughput, regardless of how many cores or CUDA threads are used.

A useful consequence is that, for a bandwidth-bound operation where we know exactly how much data must be read, measuring time directly gives us the *effective memory bandwidth* the implementation achieved:

$$\text{effective bandwidth} \approx \frac{n^3 \times 4\,\mathrm{B}}{\text{time}}.$$

This is what the size-scaling results below expose.

4 GPU Implementation (CUDA)

4.1 Parallelization strategy

The key observation is that every output element $Y[b, c]$ is an independent dot product. The natural GPU mapping is therefore *one thread per output element*, with the contracted dimension handled by a serial loop within each thread, although this is not the only viable implementation. However, this version has the most straightforward structure.

We use 32×32 thread blocks (1024 threads per block). For cases where the contracted mode is K (cases $(3, 2)$ and $(3, 1)$), both block dimensions cover the two free output dimensions. For the remaining four cases, one block dimension covers the non-K free dimension and the other covers a chunk of K (see Section 4.3). The inner loop length equals the contracted mode dimension (e.g., I for cases contracting mode 1).

For case (m=2, n=1), `threadIdx.x` is mapped to i so that consecutive threads in a warp read consecutive entries $X[i+j \cdot I+k \cdot IJ]$ — a stride-1 access that satisfies the GPU's coalescing requirement.

4.2 Engineering challenge 1: data generation on the GPU

The initial implementation generated random data on the GPU using `curandState_t`. Each state occupies 48 bytes; for I=J=K=1000, the state array alone requires $10^9 \times 48\,\mathrm{B} = 48\,\mathrm{GB}$, far exceeding the 8 GB of available VRAM on the RTX 2080Ti.

The solution is to generate data on the CPU (in the `generate` binary) and write it to a shared binary file. The CUDA program loads X and U into host RAM with `malloc`, then copies to the device with `cudaMemcpy`. Because all three implementations read the same file, the data is guaranteed to be identical across benchmarks.

4.3 Engineering challenge 2: tensors larger than VRAM

At I=J=K=1000, the tensor X is 4 GB, which barely fits in VRAM. At n=2000, X grows to 32 GB — impossible to hold on the device at once. We solve this with K-*chunking*:

1. Query available VRAM with `cudaMemGetInfo`.
2. Compute `chunk_k` $= \lfloor \text{usable}/(IJ \cdot \texttt{sizeof(float)}) \rfloor$, the number of K-slices that fit alongside U and Y.
3. Loop over K in steps of `chunk_k`: copy one chunk of X to the device, launch the kernel, proceed.
4. Copy Y back to host in one transfer after all chunks complete.

This strategy works because K-slices of X are contiguous in column-major storage (slice k starts at offset $k \cdot IJ$), so each chunk is a single contiguous `cudaMemcpy`.

For cases $(3, 2)$ and $(3, 1)$, the contracted dimension is K, so different chunks contribute partial sums to the same output elements. The device output array Y is zeroed with `cudaMemset` before the chunk loop, and each kernel call uses `Y[...] += sum` to accumulate across chunks. All other cases write Y directly (no accumulation is needed).

4.4 Timing methodology

Wall-clock time is measured with `clock_gettime(CLOCK_MONOTONIC)` wrapping the entire chunk loop, including all `cudaMemcpy` calls and the final device-to-host copy. This matches the serial and OpenMP timing methodology, which also includes data access time (both read X from a pre-loaded host buffer).

5 OpenMP Baseline

The OpenMP implementation (`parallel/batched_ttv_omp.c`) parallelizes each case with a single directive:

```
#pragma omp parallel for collapse(2) schedule(static)
```

placed over the two outer loops. The inner loop remains serial per thread, accumulating into a thread-local scalar (no atomic operations). The loop ordering is identical to the serial code — no reordering was applied to improve cache behavior for any specific case.

This implementation is intentionally unrefined. It has not been verified for correctness against a reference output and has not been tuned (e.g., loop interchange, cache-oblivious ordering, or NUMA-aware allocation). Its purpose is to provide a lower bound on what a straightforward multi-core CPU parallelization achieves, so that the GPU's advantage can be contextualized.

6 Performance Results

6.1 Speedup by case at $n = 1000$

Figure 1 shows the speedup of OMP-32 and GPU over the serial baseline for all six cases at $I{=}J{=}K{=}1000$.

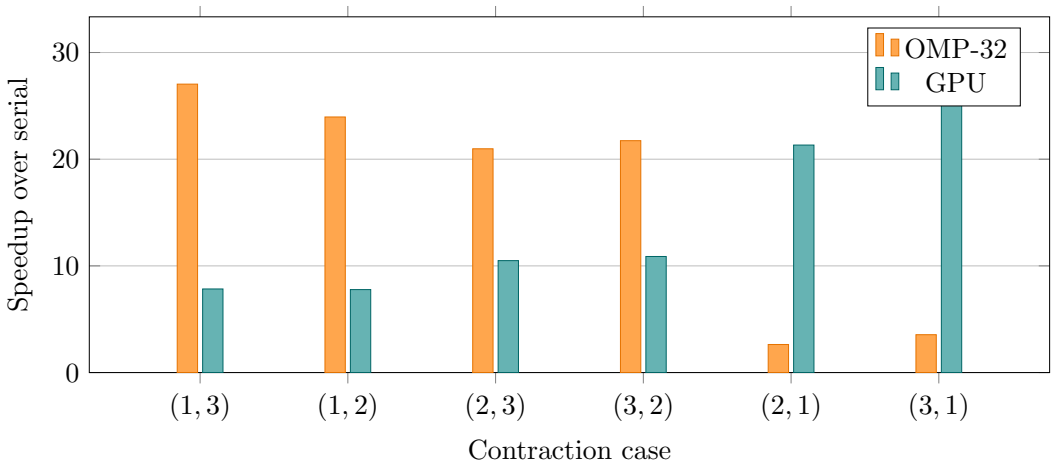

Figure 1: Speedup over serial baseline at $n = 1000$ for all six BTTV cases. OMP-32 uses 32 OpenMP threads; GPU is the CUDA implementation including all data transfers.

The OMP-32 results split cleanly along memory-access lines. Cases $(1, 3)$ and $(1, 2)$ have a stride-1 inner loop; with 32 threads the outer loops are divided evenly and the inner loop remains cache-friendly, yielding 24–27× speedup. Cases $(2, 1)$ and $(3, 1)$ have strided inner loops; every

thread in the pool issues large-stride reads, saturating memory bandwidth with 32 competing streams. The result is only 2.6–3.5× speedup despite 32 threads.

The GPU results show the opposite trend. Cases $(1, 3)$ and $(1, 2)$ achieve 7.8× speedup — real, but more modest, because these cases were already cache-friendly for the CPU. Cases $(2, 1)$ and $(3, 1)$ achieve 21–30× speedup. The GPU's high-bandwidth memory (HBM) subsystem handles the strided access pattern far better than CPU caches do; what looks like a worst case for the serial code becomes a near-best case for the GPU. Measured differently: the GPU is 8× faster than OMP-32 on case $(3, 1)$ and only 4× faster on case $(1, 3)$.

Note that GPU timing here includes the full $H \to D$ transfer of X (4 GB for $n = 1000$). The kernel execution time itself is a small fraction of the total; most of the GPU "time" is PCIe transfer.

6.2 OpenMP thread scaling

Figure 2 shows how OMP speedup grows with thread count for the best-case ($(1, 3)$) and worst-case ($(3, 1)$) memory-access cases.

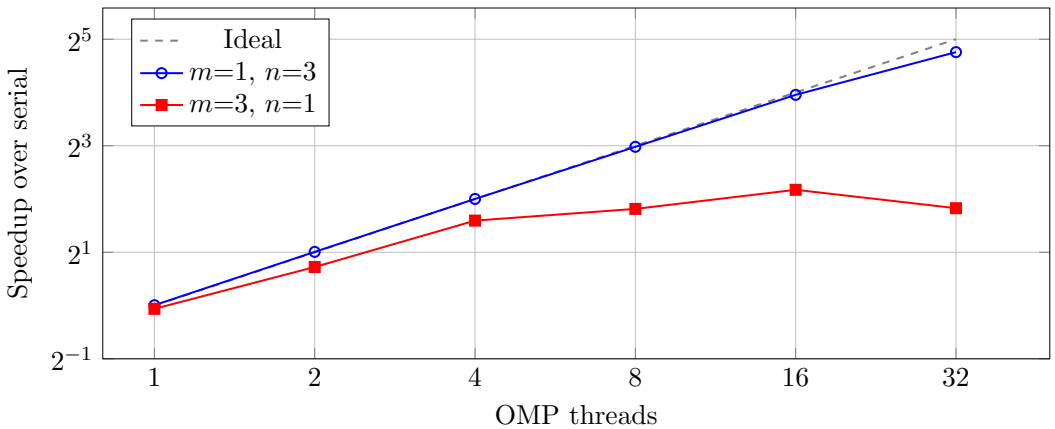

Figure 2: OpenMP speedup over serial as a function of thread count at $n = 1000$. The dashed line shows ideal linear scaling.

Case $(1, 3)$ follows near-ideal linear scaling through 32 threads (observed 27×, ideal 32×), consistent with an operation that is compute-bound per thread and cache-efficient. Case $(3, 1)$ scales only to about 3.5× at 32 threads, with diminishing returns visible beyond 8 threads. Adding more threads does not reduce the number of cache misses; it only increases contention for the memory bus. This confirms that case $(3, 1)$ is memory-bandwidth-bound even on the CPU, not merely under-parallelized.

6.3 Problem-size scaling

Figure 3 shows wall-clock time as a function of tensor mode size n (cubic $n \times n \times n$ tensors) for case (m=3, n=1), the worst-case serial scenario.

Both implementations scale as $O(n^3)$, doubling n multiplies time by roughly 8×. At n=500 the GPU runs in 0.040 s vs. 0.51 s serial (12.6×). At n=1000 the gap widens to 0.32 s vs. 9.72 s (30×). At n=2000 the serial run took 139.6 s; the GPU finished in 2.89 s (48× speedup), despite the tensor exceeding VRAM and requiring chunked transfers.

Because BTTV is bandwidth-bound (Section 3.3), these timing curves are directly interpretable as bandwidth measurements. Dividing $n^3 \times 4$ B by the measured time gives the effective throughput

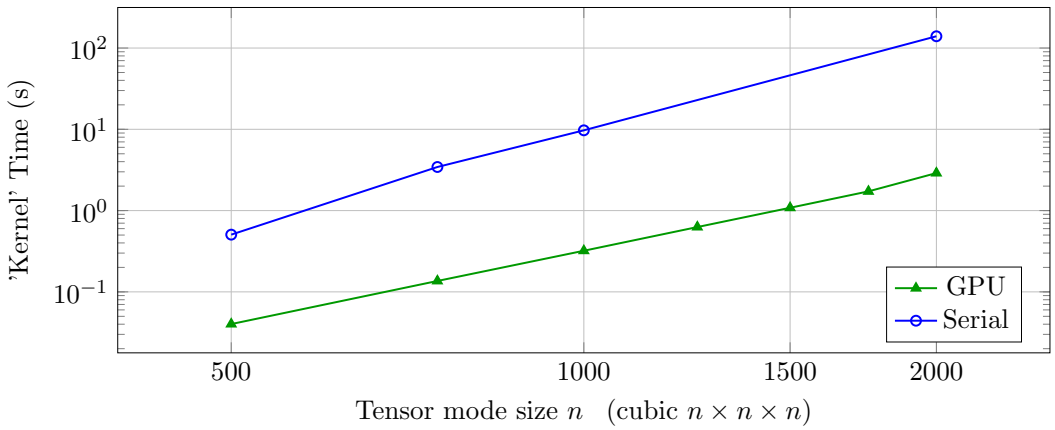

Figure 3: Wall-clock time vs. problem size for case $(3, 1)$. Serial data are missing at intermediate sizes due to run-time constraints; the n=2000 point (139.6 s) was collected separately. The GPU uses K-chunking beyond n≈1260 (where X exceeds ∼8 GB of VRAM).

at each size:

n	$\|X\|$	GPU time	GPU throughput
500	0.50 GB	0.040 s	12.45 GB/s
750	1.69 GB	0.136 s	12.39 GB/s
1000	4.00 GB	0.321 s	12.48 GB/s
1250	7.81 GB	0.627 s	12.46 GB/s
1500	13.5 GB	1.083 s	12.47 GB/s

The GPU throughput is nearly constant at ≈12.5 GB/s across all pre-chunking sizes. This is approximately the bandwidth of PCIe 3.0. The kernel finishes in milliseconds once the data is on the device; the bottleneck is moving X across the PCIe bus.

The serial effective throughput tells the opposite story. For case $(3, 1)$ the numbers are 0.99 GB/s at n=500, 0.49 GB/s at n=750, 0.41 GB/s at n=1000, and 0.23 GB/s at n=2000 — declining with n because the inner-loop stride $IJ = n^2$ grows, so each iteration jumps further in memory. At n=1000 the stride is already 4 MB, larger than any cache level, and essentially every X access is a cache miss to main memory. For the cache-friendly case $(1, 3)$ the serial throughput is flat at ≈1.2 GB/s independent of n, showing that the bottleneck there is -O0 instruction overhead rather than cache behavior.

The speedup ratios follow directly from these bandwidths. For case $(3, 1)$ at n=1000: 12.5 / 0.41 = 30.5×, matching the observed 30.3× almost exactly. For case $(1, 3)$: the GPU throughput is lower (≈9.4 GB/s, reflecting non-coalesced warp access in the kernel for this case) while serial is 1.2 GB/s, giving 9.4 / 1.2 = 7.8× — matching the observed speedup precisely. The GPU's advantage is therefore not attributable to computational power; it is a ratio of effective memory bandwidths.

The K-chunking strategy is the reason the GPU curve remains smooth beyond n≈1260 (where X alone would overflow 8 GB). Rather than failing with an out-of-memory error, the code queries free VRAM at startup, computes how many K-slices fit, and streams them through the GPU one chunk at a time. The small drop in GPU throughput at n=2000 (to 11.1 GB/s) reflects the overhead of multiple chunked transfers rather than any fundamental limit.

7 Conclusions

The batched TTV operation is embarrassingly parallel at the output-element level: every entry of Y is an independent dot product. Mapping one GPU thread to each output element exposes this parallelism directly with no communication or synchronization between threads.

The most striking finding is that the GPU's advantage scales inversely with cache friendliness. Contraction cases that are fast on a serial CPU (stride-1 inner loop) show a modest GPU speedup ($\sim$8$\times$); cases that are slow on the CPU due to strided access show a large GPU speedup ($\sim$30$\times$), because the GPU's high-bandwidth memory subsystem is far less sensitive to stride. A multi-threaded CPU implementation does not escape this problem — 32 threads make the strided cases only marginally faster because they saturate the memory bus in parallel.

The K-chunking strategy extends the GPU implementation to tensors far larger than VRAM with no correctness compromise, at the cost of multiple PCIe transfer passes. For the cases benchmarked, the dominant cost is always the host-to-device data transfer rather than kernel execution. Future work would pipeline transfers with kernel execution using CUDA streams and pinned (page-locked) host memory to reduce this overhead.

A LLM Usage Disclosure

Model: Claude Sonnet and Opus 4.6 (Anthropic), accessed via the Claude Code CLI tool throughout the development and writing of this project.

Prompts used (curated summary):

1. *Problem scoping.* Asked whether parallelizing the BTTV operation from a CP-Newton MATLAB codebase was a reasonable project scope, and whether the approach (remove MEX/BLAS, use plain C loops, port to CUDA) satisfied the course requirements. The model confirmed the fit and identified the six contraction cases and the potential for shared-memory tiling as a further optimization.

2. *MEX-to-C port.* Provided the original `batched_ttv.c` MEX source and asked for a port to a standalone C program (no MATLAB dependency, no BLAS, triple nested loops). The model produced the structure of the six case functions; all index formulas were verified manually against the mathematical definition.

3. *CUDA kernel debugging.* Shared an early draft of `batched_ttv.cu` adapted from a square matrix-multiplication example and asked the model to identify bugs. The model enumerated about ten issues (wrong variable names, 2D grid causing only the first K-slice to be initialized, incorrect timing boundary) and we worked through them one by one rather than applying them all at once. Each fix was verified by compiling and checking output.

4. *Memory architecture questions.* Asked about the `curandState_t` memory requirement and why only the first column of a small test result was nonzero. The model explained the 3D-grid bug and the 48-byte-per-state issue; both explanations matched the observed behavior and were independently confirmed.

5. *Handling tensors larger than VRAM.* Discussed whether the project was conceptually valid if problem sizes were limited to VRAM capacity. The model outlined the K-chunking strategy (`cudaMemGetInfo` $\rightarrow$ compute `chunk_k` $\rightarrow$ streaming loop), which was then implemented and confirmed working through n=2000.

6. *Report drafting.* Provided the presentation slides, source code, and benchmark results and asked for a draft of this report. The model produced the initial text; all performance numbers were cross-checked against `results.csv`, and technical descriptions were reviewed against the source code before finalizing.

7. *Bandwidth analysis.* After reviewing the draft, asked why a script computing $n^3 \times 4\,\mathrm{B}\,/\,\mathrm{time}$ reveals the memory bandwidth of the hardware. The model explained the arithmetic-intensity argument ($\mathrm{AI} \approx 0.25\,\mathrm{FLOP/byte}$ places BTTV in the bandwidth-bound regime, so time can be approximated as data/bandwidth and the inverse gives effective throughput). It identified that the constant GPU throughput ($\approx$12.5 GB/s) matches PCIe 3.0 rather than on-die HBM bandwidth, that the declining serial throughput for case $(3, 1)$ reflects cache miss rates growing with stride $IJ = n^2$, and that the observed speedup ratios match the ratio of effective bandwidths almost exactly. This analysis was incorporated into Section 3.3 and the size-scaling discussion.

Other Notes: The LLM was used for drafting, debugging guidance, and explaining unfamiliar API behavior (`curandState_t`, `cudaMemGetInfo`). Optimizations to an existing parallel algorithm were conceived by Claude. My original implementation called a modified version of the matrix multiply kernel on appropriate slices of the original tensor. This approach was rather ineffecient, so a switch was make to computing inner products directly. The text of this report was generated by the model and was edited by DuBose. I performed a first editing pass on the generated output for accuracy, re-prompted Claude to address additional concerns/details, then performed a second pass. The second pass of Claude's writing include the sections on the formulation of this problem as bandwidth-bound and estimations of what the bandwidth is with the results data.